

Excelsis 360

Data Analyst Agent

Technical Documentation · v1.1

Technical Documentation

An AI-powered data analyst for the Excelsis360 platform, built on locally-hosted LLMs, SQL Server, and a React web interface.

Version 1.1 · June 2026

Table of Contents

#	Chapter	Summary
1	Project Overview	What Excelsis 360 is, its purpose, and its key capabilities
2	Use Cases	Scenarios the system is designed to support
3	System Architecture	High-level design and data flow
4	Technology Stack	Every library and tool used and why
5	Data Backend — SQLDataStore	SQL Server integration, caching, and SQL safety
6	AI Agent — ExcelsisAgent	LangGraph ReAct loop, SqliteSaver checkpointer, and streaming
7	Agent Tools	All 13 tools exposed to the LLM
8	REST API	FastAPI endpoints, auth, rate limiting, and SSE
9	Authentication & User Management	JWT, bcrypt, and users.json
10	React Web Interface	Pages, components, and the design system
11	MCP Server	Claude Code integration via FastMCP
12	Jupyter Notebook	Interactive analysis environment
13	Configuration Reference	All environment variables
14	Security Model	What is enforced and what is not
15	Running the Stack	Installation, setup, and operation
16	Test Suite	Unit and integration tests
17	Extending the System	How to add databases, tools, and pages

1. Project Overview

The Excelsis 360 Data Analyst Agent is a full-stack AI analyst. It extends existing data infrastructure with a reasoning AI layer — combining a locally-hosted LLM (qwen2.5:14b via Ollama), direct SQL Server access, a ChromaDB RAG layer, and a dedicated web interface — to give staff a conversational way to interrogate data without writing reports or navigating dashboards.

1.1 Core Principles

- **Privacy-first.** All inference runs locally via Ollama — no data ever leaves the network.
- **Read-only data access.** AST-level SQL parsing rejects DML/DDL; database allowlist enforced.
- **Transparent reasoning.** Streaming interface shows each tool call as it happens.
- **Zero speculative statistics.** Agent never fabricates data.
- **Minimal footprint.** No cloud dependencies at inference time.

1.2 Key Features

Feature	Description
Natural-language chat	Ask questions in plain English; agent reasons and streams answer token-by-token
ReAct reasoning loop	Model decides which tools to call, in what order, across multiple steps
At-risk detection	Automatic flagging of entities below a configurable metric threshold (default 75%)
Ad-hoc SQL	Agent can write and run T-SQL SELECT queries against any configured database
Live dashboard	5 interactive Recharts charts with Power BI-style cross-filtering and drill-down
Agent ↔ dashboard link	Asking the agent to show a chart updates the dashboard live via SSE
User management	Admin-controlled accounts with bcrypt-hashed passwords and 24h JWT tokens
RAG knowledge base	ChromaDB + BAAI/bge-small-en-v1.5 (HuggingFace, auto-downloaded); indexes SQL schema and policy documents
Persistent chat history	Per-user conversation stored in SQLite via LangGraph SqliteSaver (CHAT_DB); survives restarts, shared across all Uvicorn workers
Rate limiting	10 req/min/user via SlowAPI; Redis-backed (REDIS_URI) for multi-worker accuracy; falls back to in-memory when REDIS_URI is unset
MCP server	FastMCP stdio server: ask_analyst (full ReAct loop) + 5 direct-data tools
Jupyter notebook	Full interactive analysis environment sharing the same src/ backend

2. Use Cases

Excelsis 360 is built around concrete workflows. Each use case maps to one or more agent tools.

2.1 Daily Check

A staff member asks: "Which groups had the lowest metric this week?" The agent calls `query_data(period='last_7_days')` and streams a ranked table.

2.2 At-Risk Identification

Ask: "List all entities below 70%." The agent calls `get_threshold_alerts(threshold=70.0)` and returns a table with entity IDs, labels, groups, and metric rates.

2.3 Trend Analysis

Ask: "How does this month compare to the previous month?" The agent calls `compare_periods` and returns a side-by-side table with a delta column.

2.4 Segment Comparison

Ask: "Compare segment A and B." The agent calls `compare_segments` and presents both segments side by side.

2.5 Ad-hoc Reporting

The agent calls `query_data(group_by='day_of_week')` or writes a T-SQL query via `run_sql_query` for complex custom reports.

2.6 Dashboard-Driven Exploration

Navigate to the Dashboard. Use FilterBar, click charts to cross-filter, double-click to drill down. Use 'Ask Excelsis' to open the embedded chat panel — the agent updates dashboard filters live.

2.7 Policy Knowledge

The agent calls `retrieve_policy`. The RAG layer returns top-matching excerpts from `docs/*.md` and `docs/*.pdf`. The agent synthesises a grounded answer.

2.8 Model Context Protocol (MCP)

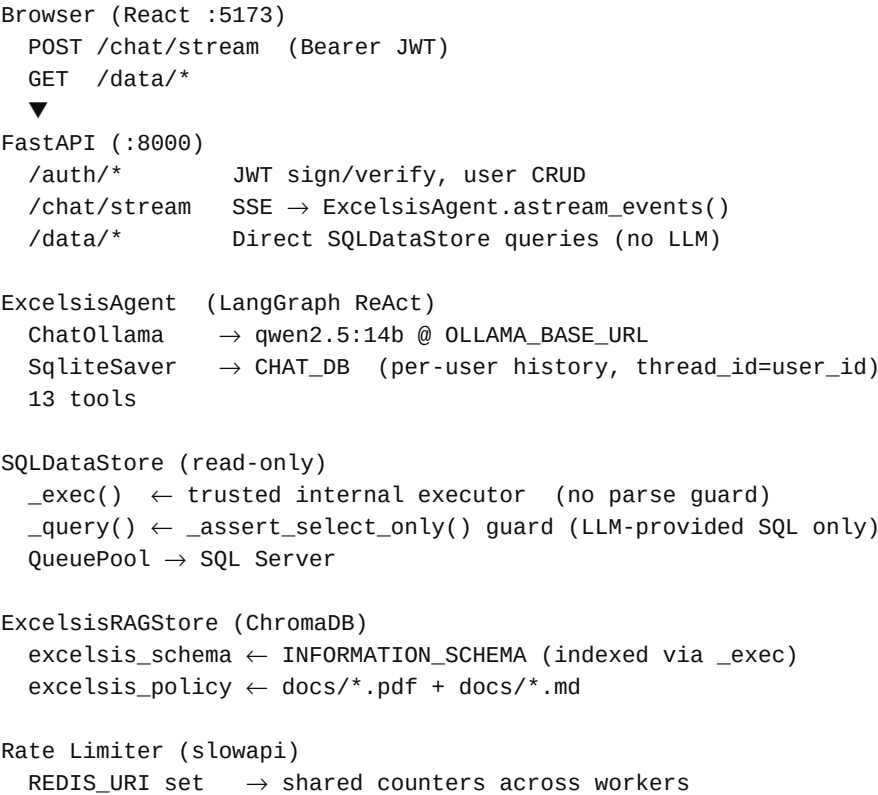
Start: `python -m src.mcp_server`. Call `ask_analyst` for full ReAct loop, or direct tools for raw results.

3. System Architecture

3.1 Request Flow

Layer	Component	Technology	Role
React App	React 18 + Vite	Browser	Renders UI, manages auth token, opens SSE stream
Fetch / Axios	Browser Fetch API	Fetch + Axios	POST /chat/stream → EventSource-style reader
FastAPI	Python / Uvicorn	FastAPI	JWT validation, rate limiting, request routing
ExcelsisAgent	LangGraph ReAct	langgraph	SqliteSaver checkpointer (CHAT_DB); streams LLM events
qwen2.5:14b	Ollama	langchain-ollama	Reasons about tool selection and response
13 tools	Python functions	langchain-core	SQL, stats, RAG, anomaly detection, trend analysis
SQLDataStore	SQLAlchemy Pool	pyodbc + sqlalchemy	Read-only T-SQL with TTL cache and _exec/_query boundary
SQL Server	ODBC Driver 18	pyodbc	Primary data table + optional extra databases

3.2 Component Diagram



REDIS_URI empty → per-process in-memory counters

3.3 Streaming Architecture

SSE Event	Frontend Action
<code>{"type": "token", "content": "..."} </code>	LLM token – appended to current message bubble
<code>{"type": "tool_start", "tool": "..."} </code>	Tool call started – spinner badge shown
<code>{"type": "tool_end", "tool": "..."} </code>	Tool call returned – spinner removed
<code>{"type": "tool_data", "tool": "...", ...} </code>	Tool returned table – rendered as interactive data grid
<code>{"type": "dashboard_filter", ...} </code>	<code>update_dashboard_view</code> called – dashboard state updates live
<code>{"type": "error", "message": "..."} </code>	Timeout or model error – displayed to user
<code>{"type": "done"} </code>	Stream complete – input field unlocked

4. Technology Stack

Library	Package	Layer	Why chosen
qwen2.5:14b	Ollama	LLM	Strong tool-calling; fits on consumer GPU or Apple Silicon
LangGraph	langgraph	Agent framework	ReAct loop with checkpointer, tool routing, async event streaming
SqliteSaver	langgraph-checkpoint-sqlite	Conversation state	Per-user persistent history via thread_id; worker-safe SQLite
langchain-ollama	langchain-ollama	LLM bridge	ChatOllama wraps Ollama's HTTP API
FastAPI	fastapi	Web framework	Async, schema-validated, auto-docs, SSE
pyodbc	pyodbc	SQL driver	ODBC 18 for SQL Server; Windows Auth + SQL Auth
SQLAlchemy	sqlalchemy	Connection pool	QueuePool per database; pool_pre_ping; contextlib.closing
sqlglot	sqlglot	SQL parser	AST SELECT-only enforcement via _assert_select_only()
pandas	pandas	Data wrangling	DataFrames for all intermediate computation
python-jose	python-jose	JWT	HS256 signing/verification; 24-hour TTL
bcrypt	bcrypt	Password hashing	Per-password salt; stored in users.json
SlowAPI	slowapi	Rate limiting	10 req/min/user; Redis-backed when REDIS_URI is set
redis	redis	Cache backend	Optional; shared rate-limit counters across workers
ChromaDB	chromadb	Vector store	Persistent local vector DB; schema + policy collections
HuggingFace Embed	langchain-huggingface	Embeddings	BAAI/bge-small-en-v1.5 — auto-downloaded; no Ollama pull
FastMCP	mcp	MCP server	stdio FastMCP; 6 tools for model-direct data access
React 18	react	Frontend	Hooks-based SPA; Vite build and HMR
Tailwind CSS v4	tailwindcss	Styling	Utility-first; custom design tokens
Recharts	recharts	Charts	Responsive SVG: bar, line, donut, area
Axios	axios	HTTP client	Auto-attaches JWT; 401 → redirect to login

Library	Package	Layer	Why chosen
React Router v6	react-router-dom	Client routing	SPA navigation; ProtectedRoute enforces auth

5. Data Backend — SQLDataStore

SQLDataStore is the single source of truth for all Excelsis360 data. It is a read-only wrapper around SQL Server in `src/sql_store.py`.

5.1 Database Schema

All column names are configurable via environment variables:

Env Var	Default	Purpose
PRIMARY_TABLE	attendance	Main table the agent queries
METRIC_COLUMN	status	Column holding the measured metric
POSITIVE_VALUE	active	Value counted as a positive outcome
DATE_COLUMN	date	Date column for time-based filtering
ENTITY_COLUMN	entity_id	Primary entity key
ENTITY_NAME_COLUMN	entity_name	Human-readable entity label
GROUP_COLUMNS	(empty)	Comma-separated grouping dimensions

5.2 SQL Safety — `_exec` vs `_query`

Two execution methods with a clear security boundary:

- **`_exec(sql, params, database)`** — trusted internal executor used by all internal methods (summary, compute_stats, get_threshold_alerts, etc.). No parse-time guard — avoids overhead on hardcoded trusted SQL. Uses `contextlib.closing(engine.raw_connection())` for safe pool lifecycle management.
- **`_query(sql, params, database)`** — security wrapper. Calls `_assert_select_only(sql)` first (sqlglot AST guard), then delegates to `_exec`. Used **exclusively** by the `run_sql_query` tool for LLM-provided SQL.

`_assert_select_only` rejects: INSERT, UPDATE, DELETE, DROP, CREATE, ALTER, TRUNCATE, MERGE, Command (EXEC/xp_cmdshell), multiple statements, and queries against databases not in `SQL_DATABASES`.

The RAG ingestor queries `INFORMATION_SCHEMA.COLUMNS` via `_exec` directly — bypassing the guard that would otherwise block system schema access.

SQL Injection Prevention

- All user-supplied values are parameterised via `pyodbc` — never string-interpolated.
- Group-by column expressions come from a hardcoded allow-list dict (`_group_expr`).
- sqlglot AST check catches multi-statement attacks even if parameterisation were bypassed.

5.3 TTL Cache

In-process `_TTLCache` with 5-minute expiry. All query methods check before hitting SQL Server. Cache key encodes all parameters so different filter combinations are cached independently.

5.4 Public Methods

Method	Description
<code>summary()</code>	Returns dict: <code>total_records</code> , <code>entity_count</code> , <code>date_range</code> , <code>metric_rate</code> , <code>below_threshold_count</code> , <code>dimensions</code>
<code>compute_stats(group_by, period, segments, ...)</code>	DataFrame aggregated by dimension and period; used by <code>query_data</code> , <code>compare_periods</code> , and <code>/data/stats</code>
<code>get_threshold_alerts(threshold, segments, ...)</code>	DataFrame of entities below threshold, ordered by <code>metric_rate</code> ascending
<code>entity_weekly_rates(entity_ids, weeks)</code>	Dict mapping <code>entity_id</code> → weekly metric rate list; used for sparkline charts
<code>analyze_weekly_trend(...)</code>	Returns direction (improving/declining/stable), <code>slope_per_week</code> , and weekly breakdown
<code>compute_statistical_summary(group_by, ...)</code>	<code>describe()</code> stats of <code>metric_rate</code> across groups
<code>detect_anomalies(group_by, sigma, ...)</code>	Groups deviating more than sigma std deviations from the mean with z-scores
<code>get_top_n(group_by, n, ascending, ...)</code>	Top or bottom N groups ranked by <code>metric_rate</code>

5.5 Connection Management

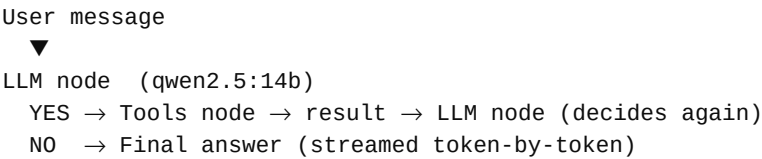
SQLAlchemy `QueuePool` (one engine per database, created at startup). Pool size = `SQL_POOL_SIZE` (default 5). Each query uses `contextlib.closing(engine.raw_connection())` to borrow from the pool and return automatically. `store.close()` disposes all engines on FastAPI lifespan shutdown.

6. AI Agent — ExcelsisAgent

ExcelsisAgent wraps a LangGraph ReAct agent powered by qwen2.5:14b via Ollama.

6.1 The ReAct Loop

ReAct interleaves Reasoning (what to do) and Acting (calling a tool). LangGraph alternates llm ↔ tools nodes until the model emits a final AIMessage with no further tool calls.



6.2 System Prompt

Fixed system prompt defines persona, tool usage rules, and analytical approach:

- Never fabricate statistics — say so if data is unavailable
- Call tools immediately — never narrate a tool call before making it
- T-SQL syntax only; SELECT-only; use TOP, DATEPART, FORMAT, CONVERT, ISNULL
- Call update_dashboard_view when user asks to see a chart or visual
- Answer greetings and general questions directly without calling tools

6.3 Conversation History — SqliteSaver

Conversation history is persisted using LangGraph's **SqliteSaver** checkpointer, stored in CHAT_DB (default ./chat.db). Each user's thread is keyed by thread_id=user.user_id in the graph config. The checkpointer loads and saves message state automatically on every call — no manual history dict, no eviction loop, no per-message append code. History survives server restarts and is shared across all Uvicorn workers.

6.4 Timeout Handling

Both ask() (sync, daemon thread + queue.Queue) and astream_events() (async, asyncio.timeout) enforce a 240-second timeout. The frontend SSE timeout is 270 s to match this plus a 30 s network buffer.

6.5 LLM Configuration

Parameter	Value / Description
model	qwen2.5:14b (overridden by MODEL env var)
base_url	http://localhost:11434 (overridden by OLLAMA_BASE_URL)
temperature	0.1 — low for consistent, factual responses

Parameter	Value / Description
num_ctx	8192 tokens context window
keep_alive	10 minutes — model stays loaded in VRAM between requests

7. Agent Tools

13 tools in `src/tools.py`, each decorated with `@tool` from LangChain. Tools returning tabular data use `response_format="content_and_artifact"` for UI table rendering.

Tool	Parameters	Description
<code>query_data</code>	<code>group_by, period</code>	Metric stats grouped by dimension and period. Calls <code>store.compute_stats()</code> .
<code>get_threshold_alerts</code>	<code>threshold=75.0</code>	Entities below threshold ordered by <code>metric_rate</code> ascending.
<code>get_summary</code>	<code>(none)</code>	High-level overview: <code>total_records</code> , <code>entity_count</code> , <code>date_range</code> , <code>metric_rate</code> , <code>dimensions</code> .
<code>update_dashboard_view</code>	<code>segments, period, view</code>	Emits <code>dashboard_filter</code> SSE event to update React state without page reload.
<code>run_sql_query</code>	<code>sql, database=""</code>	Ad-hoc T-SQL SELECT via <code>_query()</code> guard. Auto-adds TOP 200. Returns up to 200 rows.
<code>compare_periods</code>	<code>period_a, period_b</code>	Side-by-side metric rate table with delta column for two time periods.
<code>compare_segments</code>	<code>segment_a, segment_b</code>	Metric statistics for two segment values side by side.
<code>retrieve_schema</code>	<code>query</code>	Vector search of <code>excelsis_schema</code> ChromaDB collection (top-6 chunks). Use before SQL.
<code>retrieve_policy</code>	<code>query</code>	Vector search of <code>excelsis_policy</code> ChromaDB collection (top-4 chunks).
<code>statistical_summary</code>	<code>group_by</code>	Distribution stats (mean, std, min, percentiles, max) of <code>metric_rate</code> across groups.
<code>detect_anomalies</code>	<code>group_by, sigma=2.0</code>	Groups deviating more than sigma std deviations from the mean, with z-scores.
<code>get_top_n</code>	<code>group_by, n=10, ascending</code>	Top or bottom N groups ranked by <code>metric_rate</code> .
<code>analyze_trend</code>	<code>(none)</code>	Week-over-week trend: direction (improving/declining/stable), slope, weekly breakdown.

8. REST API

FastAPI backend at <http://localhost:8000>. Interactive docs at <http://localhost:8000/docs>. All endpoints except `/auth/login` and `/health` require a valid Bearer JWT.

8.1 Endpoints

Method	Path	Auth	Description
POST	<code>/auth/login</code>	None	Username + password → JWT (24h TTL)
GET	<code>/auth/me</code>	Any	Current user's username
GET	<code>/auth/users</code>	Admin	List all registered usernames
POST	<code>/auth/users</code>	Admin	Create a new user account
DELETE	<code>/auth/users/{username}</code>	Admin	Delete a user (cannot delete 'admin')
POST	<code>/chat/stream</code>	Any	SSE streaming chat → <code>ExcelsisAgent.astream_events()</code>
GET	<code>/data/summary</code>	Any	Data overview
GET	<code>/data/alerts</code>	Any	Entities below metric threshold
GET	<code>/data/stats</code>	Any	Aggregated stats by group/period
GET	<code>/data/trends</code>	Any	Current + prior 30-day weekly stats
GET	<code>/data/sparklines</code>	Any	Weekly rates for entity IDs (ids= CSV)
GET	<code>/health</code>	None	<code>{"status": "ok", "rag_ready": bool}</code>

8.2 Rate Limiting

`POST /chat/stream` is limited to 10 req/min per user (authenticated) or IP (unauthenticated). Set `REDIS_URI` to share counters across Uvicorn workers. Leave empty for per-process in-memory counters (single-worker). Returns HTTP 429 + Retry-After on exceed.

8.3 CORS

Configured via `ALLOWED_ORIGINS` (default: `localhost:5173 + localhost:3000`). In production, serve the built React app via FastAPI `StaticFiles` — CORS becomes unnecessary.

8.4 Application Lifecycle

On startup: `ensure_default_admin()`; instantiate `SQLDataStore`, `ExcelsisRAGStore`, and `ExcelsisAgent` (with `SqliteSaver` checkpointer); start background RAG ingestion thread; validate Ollama + SQL Server connectivity; refuse start if `JWT_SECRET` is the insecure default.

9. Authentication & User Management

9.1 User Store

Users stored in `api/users.json` as `username` → `bcrypt` hash. File writes use atomic write-then-rename to prevent corruption.

```
{
  "admin": { "hashed_password": "$2b$12$..." },
  "ms_johnson": { "hashed_password": "$2b$12$..." }
}
```

9.2 JWT Tokens

HS256 JWTs signed with `JWT_SECRET`. Claims: **sub** (username) and **exp** (24h). `decode_token()` reconstructs `UserContext` on every authenticated request. No refresh flow.

9.3 UserContext

Minimal dataclass with `user_id` field. Used as `thread_id` for `SqliteSaver` history. Supports per-user data filtering if added to store methods.

9.4 Admin Operations

Only admin can access `/auth/users` endpoints. Admin cannot be deleted. No password reset flow — delete and recreate to reset.

10. React Web Interface

React 18 SPA built with Vite + Tailwind CSS v4. Two-column layout (sidebar + main content).

10.1 Pages

Page	Access	Description
Login (/login)	Public	POSTs to /auth/login, stores JWT in localStorage
Chat (/chat)	Authenticated	Streams agent responses token-by-token with tool-use badges
Dashboard (/dashboard)	Authenticated	KPI cards + 5 charts + at-risk table + drilldown. Embedded chat updates filters live
Users (/users)	Admin only	List, create, and delete user accounts

10.2 Key Components

Component	Description
Sidebar	Left navigation: Chat, Dashboard, Users (admin), logout
MessageBubble	Renders chat turn with markdown, tool badges, and interactive data tables
ChatPanel	Embedded chat on Dashboard; passes dashboard_filter events to parent
FilterBar	Group multi-select + period dropdown; triggers fresh data fetch on change
Breadcrumb	Overview → Group → Entity drill navigation
DrilldownPanel	Group or entity detail with sparkline trend charts
MetricByGroupChart	Horizontal bar; single-click cross-filters, double-click drills down
WeeklyTrendChart	Line chart; click a week to filter all dashboard data
MetricBreakdownChart	Donut of positive/negative outcome proportions
WeekdayBarChart	Bar chart of metric rate by day of week
TrendComparisonChart	Grouped bar: current vs prior 30-day periods
ProtectedRoute	HOC redirecting unauthenticated users to /login

10.3 Design System

Token	Hex	Usage
carbon	#0d0d0d	Primary text, headings, icons
snow	#ffffff	Page backgrounds, card surfaces

Token	Hex	Usage
fog	#f9f9f9	Sidebar, secondary backgrounds
arctic-mist	#ececec	Borders, dividers, hover backgrounds
pewter	#5d5d5d	Secondary text, placeholders
link-blue	#007aff	Focus rings, interactive accents

10.4 SSE Client

Uses Fetch API + ReadableStream (not EventSource — which can't POST). `streamChat()` in `web/src/api/client.ts` reads chunks, splits on newlines, strips 'data: ' prefix. Timeout: 270 s (matches 240 s backend + 30 s buffer). Auth header via shared `getAuthHeader()` helper — single `localStorage` read.

11. MCP Server

stdio-based FastMCP process. Initialises SQLDataStore, ExcelsisRAGStore, and ExcelsisAgent (with SqliteSaver checkpointer). Identity fixed via MCP_USER_ID.

11.1 Exposed Tools

Tool	Description
ask_analyst(query)	Full ReAct loop: agent selects tools, reasons, and returns a complete answer
data_summary()	JSON overview: record count, entity count, date range, overall metric rate
threshold_alerts(threshold)	Entities below threshold as a formatted table
group_statistics(group_by,period)	Metric stats grouped by a dimension and period
schema_lookup(query)	Vector search of table/column metadata — use before writing SQL
knowledge_lookup(query)	Vector search of policy and rule documents

11.2 Starting the MCP Server

```
MCP_USER_ID=ms_johnson python -m src.mcp_server
# Or with default user:
python -m src.mcp_server
```

12. Jupyter Notebook

Excelsis.ipynb shares the same src/ Python backend as the web application.

12.1 Notebook Cells

Cell	Name	Description
1	Setup & imports	Load env vars, import src modules
2	Connectivity check	Verify Ollama is running; fail fast if not
3	Connect to SQL	Instantiate SQLDataStore and print summary
4	Data summary	Call store.summary() and display as dict
5	Set identity	CURRENT_USER = UserContext(user_id='...')
6	Agent setup	Instantiate ExcelsisAgent(store=store, rag_store=rag_store)
7	Chat interface	agent.ask(query, user=CURRENT_USER)
8	Direct tool calls	Call individual tools bypassing the LLM
9	Visualisations	Plotly/matplotlib charts from store data

13. Configuration Reference

Variable	Required	Default	Description
SQL_SERVER	Yes	—	SQL Server hostname, IP, or named instance
SQL_DATABASES	Yes	—	Comma-separated list of allowed databases
SQL_USERNAME	sql auth	—	SQL Server login username
SQL_PASSWORD	sql auth	—	SQL Server login password
SQL_PRIMARY_DB	No	first in SQL_DATABASES	Default database for queries
SQL_AUTH_METHOD	No	sql	sql windows azure_ad
SQL_DRIVER	No	{ODBC Driver 18 for SQL Server}	ODBC driver string
SQL_POOL_SIZE	No	5	SQLAlchemy QueuePool base connection count per database
SQL_QUERY_TIMEOUT	No	30	Per-query connection timeout in seconds
JWT_SECRET	Yes (prod)	change-me-in-production	JWT signing key — MUST be changed before production
ADMIN_PASSWORD	No	admin123	Initial admin password (first startup only)
CHAT_DB	No	./chat.db	SQLite file for LangGraph SqliteSaver conversation history
REDIS_URI	No	(empty)	Redis URI for shared rate-limit counters; in-memory fallback when unset
MODEL	No	qwen2.5:14b	Ollama model for the ReAct agent
OLLAMA_BASE_URL	No	http://localhost:11434	Ollama server base URL
OLLAMA_API_KEY	No	(empty)	Bearer token for authenticated Ollama instances
AT_RISK_THRESHOLD	No	75.0	Default metric % threshold for flagging
MCP_USER_ID	No	mcp_user	Username used by the MCP server process
PRIMARY_TABLE	No	attendance	Primary SQL table
METRIC_COLUMN	No	status	Column holding the measured metric
POSITIVE_VALUE	No	active	Value counted as a positive outcome
DATE_COLUMN	No	date	Date column for time-based filtering

Variable	Required	Default	Description
ENTITY_COLUMN	No	entity_id	Primary entity key column
ENTITY_NAME_COLUMN	No	entity_name	Human-readable entity name column
GROUP_COLUMNS	No	(empty)	Comma-separated grouping dimension columns
CHROMA_PATH	No	.chroma	Persistent ChromaDB directory path
EMBED_MODEL	No	BAAI/bge-small-en-v1.5	HuggingFace embedding model — auto-downloaded on first run
DOCS_PATH	No	docs	Directory scanned for policy PDFs and Markdown files
RAG_CACHE_TTL	No	3600	TTL in seconds for RAG query result cache
MAX_MESSAGE_LEN	No	2000	Maximum characters per chat message
MAX_PROMPT_TOKENS	No	2048	Maximum estimated tokens before rejection
ALLOWED_ORIGINS	No	http://localhost:5173, ...	Comma-separated CORS allowed origins
MLFLOW_TRACKING_URI	No	(empty)	MLflow server URI; empty = tracking disabled

14. Security Model

14.1 What Is Enforced

- **SQL read-only enforcement** — sqlglot AST rejects non-SELECT statements; applied exclusively to LLM-provided SQL via `_query()`.
- **Internal/external boundary** — hardcoded internal SQL uses `_exec()` (no overhead); LLM-provided SQL must pass `_query()` with the sqlglot guard.
- **Database allowlist** — only `SQL_DATABASES` databases can be queried.
- **SQL injection prevention** — user-supplied values parameterised; group-by from hardcoded allow-list.
- **JWT authentication** — all endpoints except `/auth/login` and `/health` require a valid JWT.
- **Password hashing** — bcrypt with per-password salts.
- **Rate limiting** — 10 req/min/user; Redis-backed across workers when `REDIS_URI` is set.
- **Local inference** — all LLM inference on localhost via Ollama; data never leaves the network.
- **Timeout enforcement** — 240 s backend + 270 s frontend SSE timeout.
- **Startup secret check** — server refuses to start if `JWT_SECRET` is still the default.

14.2 What Is Not Enforced

Known Limitations

- No row-level security — all authenticated users see all data.
- `users.json` is not encrypted — protect with filesystem permissions (`chmod 600`).
- No HTTPS — put Nginx or Caddy in front with TLS termination in production.
- No audit log — no record of who asked what or ran which SQL queries.
- No CSRF protection — stateless JWT API; tighten CORS allow-list in production.

15. Running the Stack

15.1 Prerequisites

- Python 3.11 or later
- Node.js 18 or later + npm
- Ollama installed and running (<https://ollama.com>)
- SQL Server accessible from the host machine
- ODBC Driver 18 for SQL Server installed on the host

15.2 First-Time Setup

```
# 1. Pull the LLM (one-time)
ollama pull qwen2.5:14b          # ~8 GB
# BAAI/bge-small-en-v1.5 embeddings are auto-downloaded from HuggingFace on first run

# 2. Configure environment
cp .env.example .env
# Edit .env: SQL_SERVER, SQL_DATABASES, SQL_USERNAME, SQL_PASSWORD, JWT_SECRET

# 3. Python virtual environment
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.lock

# 4. Frontend
cd web && npm install && cd ..
```

15.3 Starting Both Servers

```
bash start.sh
# FastAPI → http://localhost:8000
# React   → http://localhost:5173
```

15.4 Multi-Worker Deployment

```
# SqliteSaver writes to CHAT_DB – all workers share history
# Set REDIS_URI to share rate-limit counters:
REDIS_URI=redis://localhost:6379 uvicorn api.main:app --workers 4
```

16. Test Suite

Tests in `tests/test_qa.py`. Two classes:

16.1 Unit Tests — TestZeroKnowledge

Call tools directly with a synthetic `SampleDataStore`. No Ollama, no SQL Server, no network. Runs in under a second.

```
pytest tests/test_qa.py -v
```

16.2 Integration Tests — TestModelStress

Require a live Ollama instance with `qwen2.5:14b`. Verify complex queries complete within 120 s and greetings do not trigger tool calls.

```
pytest tests/test_qa.py -v -m integration
pytest tests/test_qa.py -v --run-all
```


17. Extending the System

17.1 Adding a New Database

- Add to SQL_DATABASES in .env. The RAG ingestor auto-indexes its INFORMATION_SCHEMA on next startup.
- The agent can immediately query it via `run_sql_query(sql=..., database='new_db')`.

17.2 Adding a New Agent Tool

```
@tool
def my_tool(param: str, config: RunnableConfig = None) -> str:
    """Describe the tool – shown to the LLM."""
    store = _store(config)
    ...
    return result_string
```

- Add to ALL_TOOLS in `src/tools.py`.
- Add description to the Tool usage section of SYSTEM_PROMPT in `src/agent.py`.

17.3 Changing the LLM

Set MODEL in .env to any Ollama model supporting tool calling. Alternatives: llama3.1:8b (faster), mistral-small (smaller context).

17.4 Adding a New Frontend Page

- Create `web/src/pages/MyPage.tsx` following existing page patterns.
- Add a route in `web/src/App.tsx` and a link in `Sidebar.tsx`.
- If data is needed, add an endpoint in `api/routers/` and include `_router()` in `api/main.py`.

17.5 Adding Per-User Data Filtering

UserContext (user_id) is available to all agent calls. To add filtering:

- Add a user→group mapping table to SQL Server.
- Read user_id from the store or rag_store references in the tool's config dict.
- Pass allowed groups as an additional filter to store methods.

17.6 Production Deployment

- Set JWT_SECRET: `openssl rand -hex 32`
- Set ADMIN_PASSWORD to a strong password.
- Build React: `cd web && npm run build`. Mount via FastAPI StaticFiles.
- Set CHAT_DB to a persistent path: e.g. `CHAT_DB=/data/chat.db`
- Set REDIS_URI for shared rate limiting: `REDIS_URI=redis://localhost:6379`

- Run: `uvicorn api.main:app --workers 4` — all workers share SqliteSaver history and Redis rate counters.
- Put Nginx or Caddy in front for TLS.
- `chmod 600 api/users.json`